

AI-WeatherWise

Project Summary

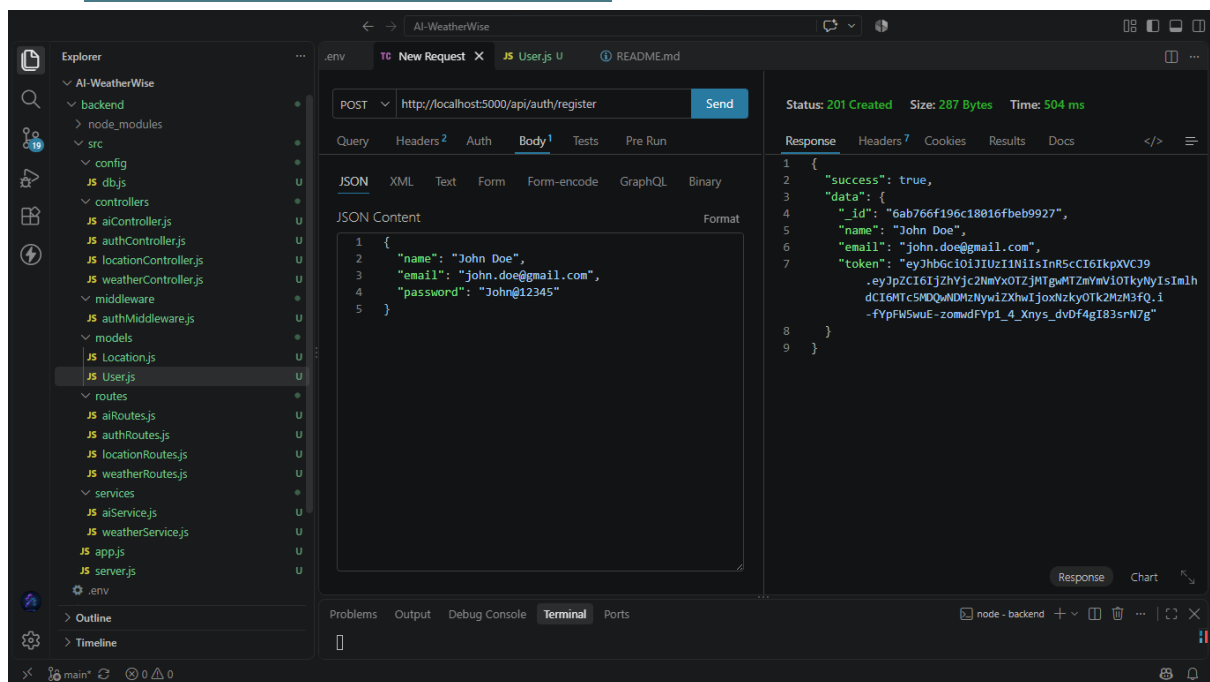
AI WeatherWise is a robust REST API-based weather application that provides real-time weather information along with AI-powered weather summaries and activity recommendations. The application is built using Node.js, Express.js, MongoDB, Mongoose, JWT, bcrypt, Google Gemini AI, and OpenWeatherMap API. Users can register and log in securely, save their favorite cities, retrieve weather information, and receive AI-generated insights and recommendations based on current weather conditions.

Project Members

1. Dharani M
2. Pavithra S
3. Thenmozhi R

1. User Authentication

POST <http://localhost:5000/api/auth/register>



The screenshot shows the VS Code interface with a REST client request and response. The Explorer sidebar on the left shows the project structure, with `JS User.js` selected. The main editor area displays a REST client request for `POST http://localhost:5000/api/auth/login`. The request body is a JSON object with `email` and `password` fields. The response is a 200 OK status with a JSON body containing `success`, `data` (with `id`, `name`, `email`, and `token`), and a `token` field. The status bar at the bottom shows the active file is `main*` and the current mode is `main*`.

The screenshot shows the VS Code interface with a REST client request and response. The Explorer sidebar on the left shows the project structure, with the `aiController.js` file selected. The main editor displays a REST client request to `http://localhost:5000/api/auth/profile` using the `GET` method. The response is a JSON object indicating success and providing user details.

Request:

```
GET http://localhost:5000/api/auth/profile
```

Response:

```
{
  "success": true,
  "data": {
    "_id": "6ab766f196c18016fbeb9927",
    "name": "John Doe",
    "email": "john.doe@gmail.com",
    "createdAt": "2026-09-26T06:32:17.131Z"
  }
}
```

POST <http://localhost:5000/api/locations>

VS Code interface showing a REST client request to `http://localhost:5000/api/locations` using the POST method. The request body is a JSON object:

```
1 {
2   "city": "Pattukottai",
3   "country": "India"
4 }
```

The response status is **201 Created**, with a size of 217 Bytes and a time of 164 ms. The response body is a JSON object:

```
1 {
2   "success": true,
3   "data": {
4     "city": "Pattukottai",
5     "country": "India",
6     "user": "6ab766f196c18016fbeb9927",
7     "id": "6ab769a396c18016fbeb9929",
8     "createdAt": "2026-09-26T06:43:47.786Z",
9     "updatedAt": "2026-09-26T06:43:47.786Z",
10    "_v": 0
11  }
12 }
```

The background shows the README.md file for the AI WeatherWise API, detailing sections for User Authentication, Location Management, and Weather Fetching.

GET <http://localhost:5000/api/locations>

VS Code interface showing a REST client request to `http://localhost:5000/api/locations` using the GET method. The response status is **200 OK**, with a size of 419 Bytes and a time of 131 ms. The response body is a JSON object:

```
1 {
2   "success": true,
3   "count": 2,
4   "data": [
5     {
6       "_id": "6ab769a396c18016fbeb9929",
7       "city": "Pattukottai",
8       "country": "India",
9       "user": "6ab766f196c18016fbeb9927",
10      "createdAt": "2026-09-26T06:43:47.786Z",
11      "updatedAt": "2026-09-26T06:43:47.786Z",
12      "_v": 0
13    },
14    {
15      "_id": "6ab769e96c18016fbeb9928",
16      "city": "Chennai",
17      "country": "India",
18      "user": "6ab766f196c18016fbeb9927",
19      "createdAt": "2026-09-26T06:42:54.409Z",
20      "updatedAt": "2026-09-26T06:42:54.409Z",
21      "_v": 0
22    }
23  ]
24 }
```

PUT <http://localhost:5000/api/locations/:id>

The screenshot shows the Visual Studio Code interface. On the left, the REST client is configured with a PUT request to `http://localhost:5000/api/locations/6ab7696e96c18016fbeb9928`. The request body is a JSON object: `{ "city": "Mumbai", "country": "India" }`. The response status is 200 OK, with a size of 70 Bytes and a time of 151 ms. The response body is: `{ "success": true, "message": "Location removed from favorites", "data": {} }`. On the right, the README file is open, showing the API documentation for the AI WeatherWise API. The documentation includes sections for User Authentication, Location Management, Weather Fetching, and AI Weather Insights.

```
1 # AI WeatherWise API
88 ## API Documentation
90 ### 1. User Authentication
91 logged-in user profile details. |
92
93 ### 2. Location Management (Private / JWT required)
94 | Endpoint | Method | Access | Request Body | Description |
95 | :--- | :--- | :--- | :--- | :--- |
96 | '/api/locations' | 'POST' | Private | '{ "city", "country" }' | Save a new favorite city. |
97 | '/api/locations' | 'GET' | Private | *None* | Retrieve all saved favorite cities for the user. |
98 | '/api/locations/:id' | 'PUT' | Private | '{ "city", "country" }' | Update a favorite city by ID. |
99 | '/api/locations/:id' | 'DELETE' | Private | *None* | Delete a favorite city by ID. |
100
101 ### 3. Weather Fetching
102 | Endpoint | Method | Access | Description |
103 | :--- | :--- | :--- | :--- |
104 | '/api/weather/:city' | 'GET' | Public | Returns current weather metrics (temperature, humidity, wind speed, condition) for a city. |
105
106 ### 4. AI Weather Insights (Private / JWT required)
107 | Endpoint | Method | Access | Request Body | Description |
108 | :--- | :--- | :--- | :--- | :--- |
```

DELETE <http://localhost:5000/api/locations/:id>

The screenshot shows the Visual Studio Code interface. On the left, the REST client is configured with a DELETE request to `http://localhost:5000/api/locations/6ab7696e96c18016fbeb9928`. The request body is a JSON object: `{ "city": "Mumbai", "country": "India" }`. The response status is 200 OK, with a size of 70 Bytes and a time of 151 ms. The response body is: `{ "success": true, "message": "Location removed from favorites", "data": {} }`. On the right, the README file is open, showing the API documentation for the AI WeatherWise API. The documentation includes sections for User Authentication, Location Management, Weather Fetching, and AI Weather Insights.

```
1 # AI WeatherWise API
88 ## API Documentation
90 ### 1. User Authentication
91 logged-in user profile details. |
92
93 ### 2. Location Management (Private / JWT required)
94 | Endpoint | Method | Access | Request Body | Description |
95 | :--- | :--- | :--- | :--- | :--- |
96 | '/api/locations' | 'POST' | Private | '{ "city", "country" }' | Save a new favorite city. |
97 | '/api/locations' | 'GET' | Private | *None* | Retrieve all saved favorite cities for the user. |
98 | '/api/locations/:id' | 'PUT' | Private | '{ "city", "country" }' | Update a favorite city by ID. |
99 | '/api/locations/:id' | 'DELETE' | Private | *None* | Delete a favorite city by ID. |
100
101 ### 3. Weather Fetching
102 | Endpoint | Method | Access | Description |
103 | :--- | :--- | :--- | :--- |
104 | '/api/weather/:city' | 'GET' | Public | Returns current weather metrics (temperature, humidity, wind speed, condition) for a city. |
105
106 ### 4. AI Weather Insights (Private / JWT required)
107 | Endpoint | Method | Access | Request Body | Description |
108 | :--- | :--- | :--- | :--- | :--- |
```

3. Weather Fetching

GET <http://localhost:5000/api/weather/:city>

The screenshot shows the VS Code interface with a REST client request configured. The request is a GET to `http://localhost:5000/api/weather/Mumbai`. The response is a JSON object with weather data for Mumbai.

```
1 {
2   "success": true,
3   "data": {
4     "city": "Mumbai",
5     "temperature": 28.4,
6     "humidity": 73,
7     "windSpeed": 4.79,
8     "condition": "Clouds",
9     "isMock": false
10  }
```

The right sidebar shows the README.md file with the following content:

```
1 # AI WeatherWise API
2 ## API Documentation
3 ## 2. Location Management (Private / JWT required)
4 favorite city by ID.
5
6 ## 3. Weather Fetching
7 | Endpoint | Method | Access | Description |
8 | :--- | :--- | :--- | :--- |
9 | /api/weather/:city | GET | Public | Returns current
10 weather metrics (temperature, humidity, wind speed, condition)
11 for a city.
12
13 ## 4. AI Weather Insights (Private / JWT required)
14 | Endpoint | Method | Access | Request Body | Description |
15 | :--- | :--- | :--- | :--- | :--- |
16 | /api/ai/weather-summary | POST | Private | { "city",
17 "temperature", "humidity", "condition" } | Generates a natural
18 language weather summary.
19 | /api/ai/weather-recommendation | POST | Private | {
20 "temperature", "condition" } | Generates personalized
21 suggestions (clothing, activities).
22
23 ---
24 ## API Testing using Postman
25 To test all endpoints:
26 1. Import the 'postman_collection.json' file located in the
27 project root into your Postman application.
```

4. AI Weather Insights

POST <http://localhost:5000/api/ai/weather-summary>

The screenshot shows the VS Code interface with a REST client request configured. The request is a POST to `http://localhost:5000/api/ai/weather-summary` with a JSON body containing weather data for Chennai. The response is a JSON object with a summary of the weather in Chennai.

```
1 {
2   "success": true,
3   "summary": "Today's weather in Chennai is warm and cloudy with high
4     humidity."
5 }
```

The right sidebar shows the README.md file with the following content:

```
1 # AI WeatherWise API
2 ## API Documentation
3 ## 2. Location Management (Private / JWT required)
4 favorite city by ID.
5
6 ## 3. Weather Fetching
7 | Endpoint | Method | Access | Description |
8 | :--- | :--- | :--- | :--- |
9 | /api/weather/:city | GET | Public | Returns current
10 weather metrics (temperature, humidity, wind speed, condition)
11 for a city.
12
13 ## 4. AI Weather Insights (Private / JWT required)
14 | Endpoint | Method | Access | Request Body | Description |
15 | :--- | :--- | :--- | :--- | :--- |
16 | /api/ai/weather-summary | POST | Private | { "city",
17 "temperature", "humidity", "condition" } | Generates a natural
18 language weather summary.
19 | /api/ai/weather-recommendation | POST | Private | {
20 "temperature", "condition" } | Generates personalized
21 suggestions (clothing, activities).
22
23 ---
24 ## API Testing using Postman
25 To test all endpoints:
26 1. Import the 'postman_collection.json' file located in the
27 project root into your Postman application.
```

POST <http://localhost:5000/api/ai/weather-recommendation>

The screenshot displays the Postman application interface. The main window shows a POST request to `http://localhost:5000/api/ai/weather-recommendation` with a JSON body: `{ "temperature": 30, "condition": "Cloudy" }`. The response status is `200 OK` with a size of `112 Bytes` and a time of `248 ms`. The response body is: `{ "success": true, "recommendation": "Stay hydrated, wear light cotton clothes, and a light jacket might be handy." }`. The right sidebar shows the `README.md` file for the `AI WeatherWise API`, which includes API documentation and testing instructions.

Request Details:

- Method: POST
- URL: `http://localhost:5000/api/ai/weather-recommendation`
- Body (JSON):

```
{ 1 2   "temperature": 30, 3   "condition": "Cloudy" 4 }
```

Response Details:

- Status: 200 OK
- Size: 112 Bytes
- Time: 248 ms
- Response (JSON):

```
1 { 2   "success": true, 3   "recommendation": "Stay hydrated, wear light cotton clothes, and a light 4     jacket might be handy." }
```

API Documentation (README.md):

```
1 # AI WeatherWise API
88 ## API Documentation
105 ### 3. Weather Fetching
106 | Endpoint | Method | Access | Description |
107 | :--- | :--- | :--- | :--- |
108 | '/api/weather/:city' | 'GET' | Public | Returns current
weather metrics (temperature, humidity, wind speed, condition)
for a city. |
109
110 ### 4. AI Weather Insights (Private / JWT required)
111 | Endpoint | Method | Access | Request Body | Description |
112 | :--- | :--- | :--- | :--- | :--- |
113 | '/api/ai/weather-summary' | 'POST' | Private | `{ "city",
"temperature", "humidity", "condition" }` | Generates a natural
language weather summary. |
114 | '/api/ai/weather-recommendation' | 'POST' | Private | `{
"temperature", "condition" }` | Generates personalized
suggestions (clothing, activities). |
115 ---
116
117 ## API Testing using Postman
118 To test all endpoints:
119 1. Import the 'postman_collection.json' file located in the
project root into your Postman application.
120 2. The collection has pre-saved example payloads for
registration, login, profile queries, location management,
weather search, and AI queries.
```